# 11-1 直接使用Request对象

- 关于请求的操作，比如从URL中提取路径参数，获取查询参数，获取请求头，获取Cookie，获取请求体中的数据；这些参数和值的获取非常方便，这是因为FastAPI帮我们创造便利。
- FastAPI底层依赖Starlette，本质上是FastAPI帮我们做了一些操作，从Starlette的Request对象解析出上述各个参数。
- 所以，对于上面这些常用的请求参数，我们可以直接使用FastAPI给我们提供的工具，并且有了数据校验、类型转化、OPenAPI文档等功能。
- 当然了，你不使用FastAPI提供的便捷工具，直接从Request对象中解析数据也是可以的，但就没由数据校验、类型转化、OPenAPI文档等功能。
- 不过，有些场景，比如说获取请求的IP，请求的client host等等，那我们就必须直接使用Request对象。

示例1：使用使用Request获取常见参数

```
from fastapi import FastAPI, Request
from starlette.requests import Request


app = FastAPI()


@app.get("/{item_id}")
def hello(req: Request):
    item_id = req.path_params.get("item_id")
    page = req.query_params.get("page", 0)
    size = req.query_params.get("size", 10)
    x_token = req.headers.get("x-token")
    x_token_cookie = req.cookies.get("x_token")

    return {
        "item_id": item_id,
        "page": page,
        "size": size,
        "x-token": x_token,
        "cookie": x_token_cookie,

    }
```

示例2：使用Request获取客户端host

- 从fastapi导入 `Request`对象，或者直接从 starlette.requests导入 `Request`， 两者是一样的效果。
- 本例中，路径参数 `item_id`是用的FastAPI提供的便捷方式，所以具有类型转换、类型校验、openapi文档等附加功能。
- 第二个形参 `req` 的类型是 `Request`，那FastAPI就知道要直接从Request中获取参数，就没有上述附加功能。

```python
from fastapi import FastAPI, Request
from starlette.requests import Request


app = FastAPI()


@app.get("/item/{item_id}")
def hello(item_id: int, req: Request):
    return {
        "item_id": item_id,
        "client_host": req.client.host
    }
```

补充：直接从Request中获取请求体参数，需要使用async/await语法，在下一章我们再详细介绍。

# 11-2 直接使用Response对象

- 关于响应的操作，在前面的章节有涉及到，比如设置响应状态码，设置响应头，在路径函数内返回数据就可以当做响应体。
- 这些都是FastAPI给我们提供的便捷方式，使用了这些方式，几乎帮助我们处理了大部分常见的需求。
- 不过，在一些特殊常场景下，我们需要更加灵活的处理响应。
- 比如，我们已经使用过了，通过Response对象来设置Cookie， 设置响应头等等。
- 本质上，FastAPI借用了Starlette框架的Response对象，即我们使用的Response对象都是Starlette的。
- Response对象还有很多其他高进的用法，比如处理重定向、响应其他格式的数据：纯文本/HTML/XML/下载文件等等。

示例1：使用Response设置状态码/响应头/cookie

- 从FastAPI导入的Response，等价于从starlette.responses导入的Response
- 在路径函数中定义的形参`response`的类型是Response，FastAPI可以是被出来你要手动使用Response对象
- 但是，这样的使用方式，本质上依然使用了FastAPI的便捷方式，因为你返回的是"123"，但其实响应格式是"application/json"
- 这是因为，FastAPI的默认响应方式是JSONResponse，即所有的返回给客户端的数据都会被json序列化

```python
from fastapi import FastAPI, Response
from starlette.responses import Response
```

```
app = FastAPI()


@app.get("/")
def hello(response: Response):
    response.status_code = 201
    response.headers["x-token"] = "12345"
    response.set_cookie("token", "111111", 100)

    return "123"
```

示例2：直接返回Response对象

- Response类初始化对象一般需要四个参数
- `content`是响应体的数据，必须是支持编码的字符串。如果要返回的数据是字段，则要使用json序列化
- `status_code` 设置响应的状态码
- `headers` 设置响应头
- `media_type` 设置响应类型，比如说是json格式的数据则值为 `application/json`

```
import json
from fastapi import FastAPI, Response
from starlette.responses import Response


app = FastAPI()


@app.get("/", )
def hello():
    response = Response(
        content=json.dumps({"hello": "world"}),
        status_code=201,
        headers={"x-token": "qqqqqq"},
        media_type="application/json"
    )

    return response
```

# 11-3 默认响应方式

上节课我们知道，可以直接返回Response对象，使用起来也比较简单，只需要在实例化对象是按照要求传参即可。

但是你会发现，不同类型的响应数据，都需要手动传参，这是比较麻烦的。

其实，FastAPI提供了多种内置的响应方式，比如对于JSON格式的响应数据，提供了JSONResponse这种响应类型。

本质上JSONResponse类是Response这个类的子类，并且JSONResponse是FastAPI中的默认响应方式。

示例1：

- 此时直接返回的是字典，但FastAPI内部帮我们处理，返回的数据格式是json的。
- 想要修改内置响应方式，可修改FastAPI()中或的APIRouter()中的 `default_response_class`
- 还可以对于指定接口，通过 `response_class` 设置响应方式
- 优先级：接口 > router > app

```
from fastapi import FastAPI, APIRouter
from fastapi.responses import JSONResponse


app = FastAPI(default_response_class=JSONResponse)          # 设置全局的默认响应方式
router = APIRouter(default_response_class=JSONResponse)      # 设置APIRouter所有接口的默认响应方式


@app.get("/", response_class=JSONResponse)                  # 设置这个接口的默认响应方式
def hello():
    return {"id": 1, "name": "liixu"}
```

示例2：源码阅读

- JSONResponse是Response的子类，m默认写死 `media_type = "application/json"`
- 并重写了 `render` 方法，该方法会在示实例化响应对象时触发，用来把我们传进去的 `content` 序列化。
- 因此使用手动JSONResponse时需要保证content是可序列化的，否则报错。

```
class JSONResponse(Response):
    media_type = "application/json"

    def __init__(
        self,
        content: typing.Any,
        status_code: int = 200,
        headers: typing.Optional[dict] = None,
        media_type: typing.Optional[str] = None,
        background: typing.Optional[BackgroundTask] = None,
    ) -> None:
        super().__init__(content, status_code, headers, media_type, background)

    def render(self, content: typing.Any) -> bytes:
        return json.dumps(
            content,
            ensure_ascii=False,
            allow_nan=False,
            indent=None,
```

```
            separators=(",", ":"),
        ).encode("utf-8")
```

# 11-4 文本相关的Response

和文本相关的有两种响应方式：

- PlainTextResponse
- HTMLResponse

示例1：PlainTextResponse用来响应纯文本的数据

- PlainTextResponse是Response的子类，重写了media_type = "text/plain"，对应响应头中的`content-type: text/plain`，此时网页上显示纯文本信息。

```
from fastapi import FastAPI
from fastapi.responses import PlainTextResponse

app = FastAPI()


@app.get("/", response_class=PlainTextResponse)
def main():
    return "Hello World"
```

示例2：HTMLResponse用来响应HTML页面

- HTMLResponse是Response的子类，重写media_type = "text/html"
- 就可以直接返回HTMLResponse对象，网页渲染HTML样式。

```
from fastapi import FastAPI
from fastapi.responses import HTMLResponse

app = FastAPI()


@app.get("/")
def index():
    html_content = """
    <html>
        <head>
            <title>Some HTML in here</title>
        </head>
        <body>
            <h1>Look ma! HTML!</h1>
        </body>
    </html>
```

```
    """
    return HTMLResponse(content=html_content, status_code=200)
```

示例3：使用response_class=HTMLResponse，此时可以直接返回HTML的文本，此时网页上依然渲染HTML样式。

- 使用response_class=HTMLResponse的另一个好处是可以在openapi文档上显示返回HTML的提示。

```
from fastapi import FastAPI
from fastapi.responses import HTMLResponse

app = FastAPI()


@app.get("/", response_class=HTMLResponse)
def index():
    return """
    <html>
        <head>
            <title>Some HTML in here</title>
        </head>
        <body>
            <h1>Look ma! HTML!</h1>
        </body>
    </html>
    """
```

# 11-5 下载文件相关的Response

下载文件相关的响应类有两个：

- StreamingResponse
- FileResponse

示例1：StreamingResponse支持文件类型的操作，下载文件

```python
from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()


@app.get("/")
def index():
    def iterfile():  #
        with open("mybook.zip", mode="rb") as f:  #
            yield from f

    return StreamingResponse(iterfile(), media_type="application/zip")
```

示例2：使用FileResponse

- 更加直接，时候文件下载。

```python
from fastapi import FastAPI
from fastapi.responses import FileResponse

app = FastAPI()


@app.get("/")
def index():
    return FileResponse("mybook.zip", filename="book.zip")  # 第一个参数文件路径，filename指定下载下来的文件名
```

# 11-6 其他Response

示例1：重定向Response

```python
from fastapi import FastAPI
from fastapi.responses import RedirectResponse

app = FastAPI()


@app.get("/")
def go_to_baidu():
    return RedirectResponse("https://www.baidu.com")
```

示例2：ORJSONResponse

- ORJSONResponse是一个基于orjson序列化的响应类，它在性能上要由于JSONResponse
- 使用是需要安装 `pip3 install orjson`

```
from fastapi import FastAPI
from fastapi.responses import ORJSONResponse

app = FastAPI()


@app.get("/", response_class=ORJSONResponse)
def go_to_baidu():
    return {"id": 1, "name": "liuxu"}
```

补充：继承Response， 模仿者其他响应类，定义自己的响应类（自己实现吧）。